Quality Assurance

Zero-Mock Testing Policy I

The repository policy prohibits mock frameworks such as `unittest.mock`, `MagicMock`, and `patch` decorators (Martin 2008; Meszaros 2007). A static gate rejects those imports, and a separate inventory surfaces semantic dependency-replacement patterns for review. Tests that require external services (Ollama or public networks) use explicit `pytest.mark` markers for conditional execution; deterministic network tests use real local HTTP servers. The philosophical motivation—analogizing excessive interaction mocking to Simmons et al.'s *researcher degrees of freedom* (Simmons et al. 2011) and the preregistration remedy (Nosek et al. 2018)—is developed fully in the Zero-Mock Tradeoff discussion.

The following example, drawn from this meta-project's own suite, illustrates zero-mock compliance:

```
def test_core_module_present(self):
    # Real filesystem, real YAML parsing - no MagicMock any
    modules = discover_infrastructure_modules(REPO_ROOT)
    names = [m.name for m in modules]
    assert "core" in names, f"'core' not found in {names}"
```

Zero-Mock Testing Policy I

This test (`tests/test_meta.py::TestDiscoverInfrastructureModules::test_core_module_present`) exercises the real `discover_infrastructure_modules` function against the real repository root. There are no mock objects substituting for the directory walk, no patched YAML parsers, and no synthetic return values—the test passes only if the infrastructure modules genuinely exist and are discoverable at their expected paths.

Coverage Thresholds I

The pipeline enforces two coverage tiers:

Coverage Thresholds I

Tier	Scope	Minimum	Current	Rationale
Project	`projects/*/src/`	90%	Reported by the current test artifact	Domain code must be thoroughly validated
Infrastructure	`infrastructure/`	60%	Reported by the current test artifact	Broader shared surface

Coverage Thresholds I

These thresholds are enforced at Stage 01 of the pipeline. A project test run below its declared 90% floor fails before downstream publication stages. This manuscript deliberately does not quote an achieved coverage percentage unless a fresh coverage artifact supplies it.

Test Suite Composition I

The repository maintains three test suites:

Test Suite Composition I

- **Infrastructure tests** (`tests/`): ~9,874 tests validating the 28 infrastructure subdirectories, covering logging, rendering, validation, steganography, reporting, and LLM integration.
- **Project tests** (`projects/*/tests/`): Per-project suites whose sizes scale with each exemplar's surface area — for example 304 tests in `template_autoresearch_project` and 243 in `template_code_project`, with several exemplars larger still. (A true min/max span would require dedicated `project_test_count_min`/`project_test_count_max` tokens in `build_manuscript_metrics_dict`; see the meta-template's generator backlog.)
- **Integration tests**: Embedded within infrastructure tests, these exercise full pipeline stages against real manuscript inputs, validating end-to-end behavior from Markdown source to rendered PDF.

Visualization Standards I

All generated figures must meet accessibility requirements:

Visualization Standards I

- Shared 16pt font constant for primary text elements, with compact annotations derived from that constant.
- Colorblind-safe palettes (IBM Design / Wong palette) with high-contrast fallbacks.
- 200 DPI PNG rendering, sourced from the same constant used by every figure writer.
- Descriptive axis labels and figure titles.
- No reliance on color alone to convey information—redundant encoding via shape, pattern, or annotation is used where applicable.

The `test_architecture_viz.py` suite verifies that each real figure generator writes a non-empty PNG and that the comparative matrix has its declared shape and value domain. The font and render-resolution values above are injected from `viz_palette.py`, preventing prose and implementation from drifting even though visual accessibility still requires human inspection of rendered figures.